Low-Level Design (LLD) Roadmap for 5-Year Software Engineer

1. Object-Oriented Programming (OOP)

Strong OOP fundamentals are essential for building scalable systems.

1. Encapsulation, Abstraction, Inheritance, Polymorphism
2. Composition vs Inheritance
3. Interfaces and Abstract Classes
4. Loose coupling and high cohesion

2. SOLID Principles

These principles help in writing clean and maintainable code.

1. Single Responsibility Principle
2. Open/Closed Principle
3. Liskov Substitution Principle
4. Interface Segregation Principle
5. Dependency Inversion Principle

3. Design Patterns

Design patterns are reusable solutions to common problems.

1. Creational: Singleton, Factory, Builder
2. Structural: Adapter, Decorator, Facade
3. Behavioral: Strategy, Observer, Command

4. UML & Design Modeling

1. Class Diagrams
2. Sequence Diagrams
3. Activity Diagrams

5. LLD Practice Problems

1. Parking Lot System
2. Cab Booking System
3. ATM Machine
4. Library Management System
5. Food Delivery System

6. Java Implementation

1. Implement designs using Java
2. Use interfaces and abstraction
3. Apply design patterns in code

7. Concurrency & Multithreading

1. Threads and Runnable
2. Synchronization
3. Executor Framework
4. Thread safety

8. Database Design

1. Entity relationships
2. Normalization
3. Indexing
4. ACID transactions

9. API Design

1. REST API principles
2. Error handling
3. Versioning

10. Clean Code

1. Readable and maintainable code
2. Refactoring techniques
3. Avoid code smells

11. Testing

1. Unit Testing (JUnit)
2. Mocking (Mockito)
3. Testable architecture

Weekly Study Plan

1. Day 1-2: Learn concepts
2. Day 3-4: Solve LLD problem
3. Day 5: Code implementation
4. Day 6: Refactor
5. Day 7: Revision